

Algoritmos Recursivos em Julia

MAT4642 – Introdução à Matemática Computacional
Prof. Luiz-Rafael Santos · UFSC Blumenau · Semestre 2026.1

1 Motivação: de somatórios a recursividade

Na última aula calculamos somatórios com laços `for`. Considere:

$$S(n) = \sum_{i=1}^n i = 1 + 2 + \cdots + n.$$

Há uma outra maneira de pensar nessa soma. Observe que

$$S(n) = \underbrace{1 + 2 + \cdots + (n-1)}_{S(n-1)} + n = S(n-1) + n,$$

com o caso inicial $S(1) = 1$ (ou $S(0) = 0$). O problema de tamanho n *se reduziu* ao problema de tamanho $n-1$. Essa é a essência da **recursividade**.

Recursividade – Intuição

Um algoritmo é *recursivo* quando a solução de um problema de tamanho n é definida em termos da solução do *mesmo problema* para entradas **menores**, até atingir um **caso base** com resposta imediata e trivial.

Em matemática esse padrão é chamado de **definição por recorrência** ou **indução**.

2 Estrutura de um algoritmo recursivo

Definição: Algoritmo Recursivo

Uma função f definida por recorrência tem a forma:

$$f(n) = \begin{cases} \text{valor base}, & n = n_0 \quad (\text{caso base}), \\ g(n, f(n-1)), & n > n_0 \quad (\text{passo recursivo}). \end{cases}$$

Em Julia, isso corresponde a uma função que chama a si mesma:

```
1 function f(n)
2     if <condicao_de_parada>                # caso base
3         return <valor_base>
4     else
5         return <expressao_com_f(n-1)>    # passo recursivo
```

```

6     end
7 end

```

Atenção

Todo algoritmo recursivo precisa satisfazer **dois requisitos**:

1. **Caso base:** pelo menos uma condição na qual a função retorna diretamente, sem nova chamada recursiva.
2. **Convergência:** cada chamada recursiva deve aproximar o argumento do caso base, garantindo que a recursão *termine*.

Sem um caso base correto, a função entra em recursão infinita e o programa trava com *stack overflow*.

3 Exemplos fundamentais

3.1 Somatório

Exemplo: Somatório recursivo

A definição recursiva é:

$$S(n) = \begin{cases} 0, & n = 0, \\ S(n-1) + n, & n \geq 1. \end{cases}$$

```

1 function somatorio_rec(n)
2     if n == 0
3         return 0                                # caso base:
4         S(0) = 0
5     else
6         return somatorio_rec(n - 1) + n          # passo
7         recursivo
8     end
9 end

```

Rastreamento para $n = 4$:

$$\begin{aligned}
 \text{somatorio_rec}(4) &= \text{somatorio_rec}(3) + 4 \\
 &= (\text{somatorio_rec}(2) + 3) + 4 \\
 &= ((\text{somatorio_rec}(1) + 2) + 3) + 4 \\
 &= (((\text{somatorio_rec}(0) + 1) + 2) + 3) + 4 \\
 &= (((0 + 1) + 2) + 3) + 4 = 10.
 \end{aligned}$$

Podemos verificar com a fórmula fechada: $S(n) = \frac{n(n+1)}{2}$. Para $n = 4$: $S(4) =$

$$\frac{4 \cdot 5}{2} = 10. \checkmark$$

3.2 Fatorial

O fatorial $n!$ é definido matematicamente por recorrência:

$$n! = \begin{cases} 1, & n = 0, \\ n \cdot (n-1)!, & n \geq 1. \end{cases}$$

Exemplo: Fatorial recursivo e iterativo

```

1  # Versao recursiva
2  function fatorial_rec(n)
3      @assert n >= 0 "n deve ser nao-negativo"
4      if n == 0
5          return 1                                # 0! = 1 (caso
6              base)
7      else
8          return n * fatorial_rec(n - 1)          # n! = n * (n-1)!
9      end
10 end
11
12 # Versao iterativa (para comparacao)
13 function fatorial_iter(n)
14     @assert n >= 0 "n deve ser nao-negativo"
15     fat = 1
16     for i in 1:n
17         fat *= i
18     end
19     return fat
20 end

```

Rastreamento para $n = 3$:

$$\text{fatorial_rec}(3) = 3 \cdot \text{fatorial_rec}(2) = 3 \cdot (2 \cdot \text{fatorial_rec}(1)) = 3 \cdot (2 \cdot (1 \cdot \text{fatorial_rec}(0))) =$$

3.3 Sequência de Fibonacci

A sequência 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ... é definida por:

$$F_n = \begin{cases} 0, & n = 0, \\ 1, & n = 1, \\ F_{n-1} + F_{n-2}, & n \geq 2. \end{cases}$$

Note que aqui há **dois casos base** ($n = 0$ e $n = 1$) e o passo recursivo depende dos dois valores anteriores.

Exemplo: Fibonacci recursivo

```

1 function fibonacci_rec(n)
2     @assert n >= 0 "n deve ser nao-negativo"
3     if n <= 1
4         return n                # F(0) = 0, F(1) = 1
5     else
6         return fibonacci_rec(n - 1) + fibonacci_rec(n - 2)
7     end
8 end

```

Problema de eficiência – Fibonacci

A implementação recursiva ingênua recalcula os mesmos subproblemas repetidamente. O número total de chamadas para calcular $F(n)$ cresce *exponencialmente*: $\approx 2^n$. Para $n = 40$, são mais de 300 milhões de chamadas!

n	F_n	Chamadas recursivas
10	55	$\sim 2^{10} = 1\,024$
20	6\,765	$\sim 2^{20} \approx 10^6$
30	832\,040	$\sim 2^{30} \approx 10^9$

A versão **iterativa** usa $O(n)$ operações e memória $O(1)$:

```

1 function fibonacci_iter(n)
2     @assert n >= 0
3     if n <= 1
4         return n
5     end
6     a, b = 0, 1
7     for i in 2:n
8         a, b = b, a + b        # atualiza ambos
                                # simultaneamente
9     end
10    return b
11 end

```

3.4 Máximo Divisor Comum — Algoritmo de Euclides

O **Algoritmo de Euclides** (≈ 300 a.C.) é um dos mais antigos algoritmos formalizados. Baseia-se na propriedade:

$$\text{mdc}(a, b) = \text{mdc}(b, a \bmod b), \quad b > 0.$$

Proposição 1. Para quaisquer inteiros $a \geq b > 0$,

$$\text{mdc}(a, b) = \begin{cases} a, & b = 0, \\ \text{mdc}(b, a \bmod b), & b > 0. \end{cases}$$

Exemplo: MDC recursivo

```

1 function mdc(a, b)
2     if b == 0
3         return a                # caso base
4     else
5         return mdc(b, mod(a, b)) # passo recursivo
6     end
7 end

```

Rastreamento para $\text{mdc}(48, 18)$:

$$\text{mdc}(48, 18) \longrightarrow \text{mdc}(18, 12) \longrightarrow \text{mdc}(12, 6) \longrightarrow \text{mdc}(6, 0) = 6.$$

O número de passos é $O(\log(\min(a, b)))$ — extremamente eficiente.

4 Recursão vs. Iteração

Aspecto	Recursão	Iteração
Legibilidade	Espelha a definição matemática	Pode ser mais explícita no controle
Memória	Usa a pilha de chamadas (<i>stack</i>)	Usa variáveis locais
Risco	<i>Stack overflow</i> para n grande	Segura para n grande
Desempenho	Pode ter overhead de chamadas	Em geral mais rápida em laços simples
Aplicação natural	Estruturas hierárquicas, indução	Laços simples, acumuladores

Regra prática

Se a **definição matemática** do problema é recursiva, comece com a versão recursiva — ela é mais legível e fácil de verificar. Se o **desempenho** for crítico ou a entrada for grande, converta para a versão iterativa (ou use memoização). Em Julia, ambas as abordagens são eficientes quando bem aplicadas.

5 Exercícios

Exercício 1 – Potência recursiva

Implemente `potencia(a, n)` que calcula a^n para $a \in \mathbb{R}$ e $n \in \mathbb{N}$ usando a definição recursiva:

$$a^n = \begin{cases} 1, & n = 0, \\ a \cdot a^{n-1}, & n \geq 1. \end{cases}$$

Compare o resultado com o operador `^` nativo do Julia.

Exercício 2 – Soma dos quadrados

Implemente recursivamente:

$$Q(n) = \sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2.$$

Verifique que $Q(n) = \frac{n(n+1)(2n+1)}{6}$ comparando com sua implementação.

Exercício 3 – Exponenciação rápida

A **exponenciação rápida** (*fast exponentiation*) reduz o número de multiplicações pela metade a cada dois passos:

$$a^n = \begin{cases} 1, & n = 0, \\ (a^{n \div 2})^2, & n \text{ par}, \\ a \cdot a^{n-1}, & n \text{ ímpar}. \end{cases}$$

Implemente `pot_rapida(a, n)`. Compare o número de operações com o Exercício 1 para $n = 64$.

Exercício 4 – MMC via MDC

Use a relação $\text{mmc}(a, b) = \frac{a \cdot b}{\text{mdc}(a, b)}$ para implementar `mmc(a, b)`. Verifique com $\text{mmc}(12, 18) = 36$.

Exercício 5 – Sequência de Collatz

A **conjectura de Collatz** afirma que a sequência

$$c(n) = \begin{cases} 1, & n = 1, \\ c(n/2), & n \text{ par}, \\ c(3n + 1), & n \text{ ímpar}, \end{cases}$$

sempre termina em 1. Implemente `collatz(n)` que retorna o *número de passos* até atingir 1. Qual o valor de `collatz(27)`?

Exercício 6 – Desafio – Fibonacci iterativo

Converta `fibonacci_rec` em uma versão iterativa que use apenas $O(1)$ de memória extra. Meça o tempo de execução para $n = 40$ nas duas versões usando `@elapsed`.

Handout produzido com L^AT_EX para a disciplina MAT4642 – UFSC Blumenau, 2026.1. Notebook Pluto disponível no Moodle da disciplina.